

Overview of actions

		Action	Description	Time spent(min)
User Story	"UNet – VGG19 encoder (256×256)" - Objective: Implement a CNN used for Image Segmentation on TT-NN. Start by PyTorch prototyping and move to deployment on Tenstorrent hardware with TT-NN. This developer journey is designed to go over environment setup, dataset preparation, model design, parameter conversion, input preprocessing, TT-NN implementation of UNet components, and validation against the PyTorch baseline. Complete each step individually in a modular fashion if possible to ensure easy verifiability. As a developer you should be able to reproduce the PyTorch baseline, port the model to TT accelerators, and optimize it for accuracy, correctness, and efficiency.	Core Setup Actions:		
		Install deps Verify installation Initialize device Data & Preprocessing Actions: Define dataset interface Image sizing Augmentation (train) Normalization Batching Tiling (optional) Model Architecture Actions: Define VGG19 encoder Freeze policy Bridge layers UNet decoder Skip connections Final head Exportable model Training & Baseline (PyTorch): Losses Metrics Optimizer & sched Baseline checkpoint Sanity visualizations Parameter Conversion Actions (PyTorch → TT-NN): Extract weights Conv weight layout Bias terms BatchNorm folding Activation parity Save conversion map Load to device Input Pipeline Actions (TT-NN): Preprocess to TT Batch handling I/O staging UNet-Specific Implementation Actions (TT-NN): Implement conv blocks Downsampling Upsampling Concat skips Nonlinearities Final logits Validation & Debug Actions: Shape parity checks Numerical checks Whole-model parity Edge cases Performance Measurement Actions: Warm-up & timing Throughput & latency Memory profiling	pip install torch torchvision ttnn albumentations opencv-python (plus your experiment stack) Run a tiny smoke test: import libs, query device, simple ttnn tensor ops Open TT device(s), set mesh, precision (e.g., FP16/BF16), seeds, deterministic flags Create SegDataset reading images/masks; support train/val/test splits Center-crop/resize to 256×256 (keep aspect policy documented) Albumentations: flips, small rotations, elastic/affine, color jitter (for robustness) Per-channel mean/std (ImageNet if using pretrained VGG19); scale masks to {0,1} Implement PyTorch DataLoader with worker count/pinning; ensure mask dtype long/float For >256 inputs, optional sliding-window/overlap tiling & reassembly pipeline Use torchvision VGG19 (optionally pretrained=True), extract feature blocks at 1/2/4/8/16 scales Choose which encoder stages to freeze/unfreeze (warm-up, then unfreeze deeper blocks) 1×1 or 3×3 convs to adjust channel dims from VGG blocks to UNet skip widths 5-stage up path: Upsample (transpose conv or bilinear+3×3 conv), concat with skip, double conv Wire encoder feature maps to matching decoder stages; handle size off-by-one with padding/crop 1×1 conv → C_out classes; activation: sigmoid (binary) or softmax (multi-class) Keep clean module boundaries so you can export weights per-layer for TT-NN Binary/multi-class: BCE/BCEWithLogits + Dice; or Focal + Dice (log-cosh Dice optional) Dice, IoU (mIoU), Pixel Acc, Precision/Recall; compute per-class if multi-class AdamW/SGD; cosine/plateau sched; mixed precision (torch.cuda.amp) for speed Train to convergence on 256×256; save state_dict and per-layer shapes for conversion Save overlay grids (input/pred/mask) for a fixed validation set Walk modules in deterministic order; record (name, type, shape, dtype) Convert from PyTorch (out, in, kH, kW) to TT-NN expected layout; handle groups=1 Export/attach biases per conv/bridge/head; default to zeros if layer had no bias Fold BN into preceding conv (scale/shift using running stats) to simplify TT graph Confirm TT-NN uses same nonlinearities (ReLU, LeakyReLU); document in-place vs out-of-place Emit a JSON manifest mapping PyTorch modules → TT-NN ops, including tensor transposes Create TT tensors with correct memory layout/stride; upload to TT device(s) Apply the exact normalization as in PyTorch; convert NCHW fp32 → device dtype/layout Support 1 and N>1; document max batch that fits L1/DRAM; pad to tiles if needed Asynchronous host→device copy; pre-allocate input/output buffers for steady-state runs ttnn.conv2d (3×3, stride=1, padding=1) ×2 per stage; verify padding equivalence MaxPool2d stride=2 (or strided conv); ensure exact output sizes vs PyTorch Transposed conv 2× (preferred) or bilinear upsample + 3×3 conv; match PyTorch semantics Channel-wise concat of upsampled map + encoder skip; adjust padding/cropping to match ttnn.relu (or chosen activation); ensure no in-place mutations affecting grads (if training) 1×1 conv to C_out; optional ttnn.sigmoid/softmax done post-hoc for evaluation Print per-stage shapes (enc/dec) PyTorch vs TT-NN on a fixed input Layer-by-layer RMS/Max error; tolerate minor fp16/bf16 drift; assert thresholds Compare full forward outputs; compute Dice/IoU deltas on a small val batch Non-divisible sizes (if you allow >256), odd kernels, off-by-one crops, empty masks Discard warm-ups; measure median/percentiles over many iters (steady state) Report imgs/sec and p50/p95 latency; also host→device transfer time Peak device DRAM/L1; activation checkpointing options; workspace sizes	